Pandas, EDA, Visualisation & APIs

Session 12 study guide · pandas · seaborn · requests · VS Code

Session 11 ended with a question: what do we use when columns have **names**, values have **different types**, and some entries are **missing**? The answer is **pandas**. This guide starts with a two-page recap of everything so far, then builds up from the **Series** (one labelled column) to the **DataFrame** (a labelled table), and follows the Session 12 deck: loading files, inspecting, cleaning, combining, grouping, exploring with charts, and finally reading data from an **API**.

All examples use the session's own files: two weeks of fictional retailer transactions, a hub lookup workbook and a saved World Bank response. Every example was run with pandas 3.0 and the output shown is real. The numbers match the slides.

Part	Topic
0	Recap: data types, data structures, NumPy
1	Series: one labelled column
2	DataFrames: a labelled table
3	Loading data: CSV, Excel, JSON, SQL
4	First inspection
5	Selecting, filtering, sorting and other everyday operations
6	New columns and business rules
7	Data quality: missing values, duplicates, types
8	Combining tables and grouping
9	Exploratory analysis and visualisation
10	Working with APIs
11	Practical checkpoints

Part 0 Recap: what we have covered so far

Everything in pandas builds on these three layers. Use this as a quick reference before moving on.

Data types (Session 07)

Type	Example	When to use it
`int`	`quantity = 4`	counts and whole numbers
`float`	`unit_price = 12.40`	prices, rates, measurements
`str`	`hub = "East Hub"`, `txn = "TXN-001"`	names, labels, codes, IDs (even when they contain digits)
`bool`	`is_large = total >= 950`	yes/no flags; the result of a comparison; filter conditions
`None`	`note = None`	"no value"; pandas shows missing values as `NaN` or `<NA>`

Values from `input()`, a CSV or JSON often arrive as **text**: convert with `int()`, `float()` (or `pd.to_numeric` in pandas) before calculating.

Control flow and functions (Session 08)

Tool	Example	When to use it
`if / elif / else`	`if total >= 950: band = "large"`	choose a path from a rule
`for` / `while`	`for order in orders:`	repeat for every item / until a condition changes
function	`def apply_tax(x, pct): return x * (1 + pct/100)`	name a job once and reuse it
`try / except`	`except ValueError:`	recover from an expected failure such as bad input

Data structures (Session 09)

Structure	Example	Mutable?	When to use it
list	`[120.5, 89.99, 89.99]`	yes	ordered items, duplicates allowed; collecting items as they arrive
tuple	`("ORD-1001", "West")`	no	a small fixed group; unpacking `order_id, region = pair`
dictionary	`{"hub": "E", "units": 4}`	yes	one record with named fields; fast lookup by key
set	`{"E", "C", "W"}`	yes	unique values; membership; `&` both, `|` either, `-` difference
list of dicts	`[{"id": "A1", ...}, {"id": "A2", ...}]`	yes	many records: each dict a row. **This becomes a DataFrame.**

NumPy arrays (Session 11)

Functionality	Example	When to use it
Create	`np.array([4, 5])`, `np.arange(1, 15)`, `np.zeros((2, 3))`	numbers in a regular shape
Inspect	`a.shape`, `a.ndim`, `a.size`, `a.dtype`	check the structure before calculating
Index and slice	`d[1, :]` row · `d[:, 2]` column · `d[:7, :]` block	pick a row, a column or a block by position
Boolean mask	`d[d > 120]`	keep the values that pass a condition
Vectorise	`prices * 1.15`	one formula applied to every value, no loop
Broadcast	`dispatch / daily_capacity` ((14, 3) with (3,))	combine a grid with one value per column
Aggregate	`d.sum(axis=0)`, `d.mean()`, `d.argmax()`	totals per column or row; where the peak is
Reshape, split, stack	`reshape(14, 3)`, `np.split`, `np.vstack`	give flat data a structure; cut and rebuild

Where NumPy stops, and pandas begins

Real data needs...	NumPy	pandas
column **names**	we remember "column 0 is East"	columns are named: `df["hub_code"]`
mixed types per column	one dtype for everything	each column has its own dtype
missing values	awkward	`NaN` built in, with `isna()`, `dropna()`, `fillna()`
row **labels**	positions only	an index of labels, plus `.loc` and `.iloc`

pandas is built **on** NumPy: every numeric column is a NumPy array underneath, so vectorising, masks and `axis` all carry over.

Lesson plan (3 hours)

Time	Block	You do (trainer)	Learners do	Slides
00–08	Bridge and grain	S11 array vs one transaction record. What does one row represent?	Predict: grain; label vs position.	1–5
08–26	Parts 1–4	Series, DataFrame, CSV/Excel intake, inspection.	Checkpoint 1.	6–12
26–46	Parts 5–6	Select, filter, sort; define the order-value metric.	TODO 1.	13–18
46–66	Part 7	Missing ≠ zero; duplicates; safe type conversion.	TODOs 2–5.	19–27
66–76	Break			28
76–99	Part 8	concat vs merge; left vs inner; groupby; reconcile.	TODOs 6–9.	29–37
99–117	Part 9	Summary statistics, histogram, boxplot, honest charts.	TODO 10.	38–44
117–140	Part 10	HTTP, GET/POST, status codes, JSON, consume vs expose.	Predict: metadata or records?	45–55
140–174	Practical	Protect learner time; hints, not solutions.	TODOs 11–14.	56–58
174–180	Close	One decision each: cleaning, join, chart, API.	Self-check.	59–61

Before class

☐ Install the libraries once in the VS Code terminal: `pip install pandas openpyxl matplotlib seaborn requests` (`openpyxl` reads `.xlsx` files).

☐ A folder `s12_pandas` opened in VS Code containing the four data files: `S12_Transactions_Week1.csv`, `S12_Transactions_Week2.csv`, `S12_Hub_Reference.xlsx`, `S12_WorldBank_Population_Snapshot.json`.

☐ Learners can use the Starter notebook (VS Code opens `.ipynb` files) or plain `.py` scripts. In a script, wrap displays in `print(...)`; notebooks show the last value automatically.

☐ Keep the Solution notebook private. Test the World Bank request before class; the saved snapshot is the plan B.

Part 1 Series: one labelled column

A **Series** is a one-dimensional sequence of values **plus an index of labels**. Think of it as a NumPy array whose positions also have names, or a dictionary that can do vectorised maths.

Creating a Series

p1_series_dict.py

```
import pandas as pd

hub_names = pd.Series({"E": "East Hub", "C": "Central Hub", "W": "West Hub"})
print(hub_names)
print(hub_names["E"])
```

OUTPUT

```
E       East Hub
C    Central Hub
W       West Hub
dtype: str
East Hub
```

From a dictionary, the keys become the **index** and the values become the data. Reading `hub_names["E"]` finds a value by **label**, which an array cannot do.

> **Business case:** A shop records its sales for each day of one week.

p1_series_list.py

```
import pandas as pd

daily_sales = pd.Series([1250.0, 980.5, 1430.0, 1105.25, 1890.0],
                        index=["Mon", "Tue", "Wed", "Thu", "Fri"],
                        name="sales_usd")
print(daily_sales)
print(daily_sales.dtype, daily_sales.shape)
```

OUTPUT

```
Mon    1250.00
Tue     980.50
Wed    1430.00
Thu    1105.25
Fri    1890.00
Name: sales_usd, dtype: float64
float64 (5,)
```

From a list you can supply the labels with `index=`. `name=` gives the Series a name; inside a table that name is the column heading.

Labels and positions

p1_series_access.py

```
print(daily_sales["Wed"])        # by label
print(daily_sales.iloc[2])       # by position (0, 1, 2 ...)
print(daily_sales["Tue":"Thu"])  # a label slice INCLUDES the end
```

OUTPUT

```
1430.0
1430.0
Tue     980.50
Wed    1430.00
Thu    1105.25
Name: sales_usd, dtype: float64
```

> **Watch out:** A slice by **label** (`"Tue":"Thu"`) includes the end label; a slice by **position** (`.iloc[1:4]`) excludes the end, like lists and arrays.

Vectorised maths, filters and summaries work like NumPy

p1_series_ops.py

```
print(daily_sales * 1.15)                # add 15% VAT to every day
print(daily_sales[daily_sales > 1200])   # Boolean filter keeps the labels
print("Total:", daily_sales.sum(), " Mean:", daily_sales.mean())
print("Best day:", daily_sales.idxmax())  # the LABEL of the maximum
```

OUTPUT

```
Mon    1437.5000
Tue    1127.5750
Wed    1644.5000
Thu    1271.0375
Fri    2173.5000
Name: sales_usd, dtype: float64
Mon    1250.0
Wed    1430.0
Fri    1890.0
Name: sales_usd, dtype: float64
Total: 6655.75  Mean: 1331.15
Best day: Fri
```

Compare `idxmax()` with NumPy's `argmax()` from Session 11: NumPy gave a position (11) that we had to convert to Day 12; a Series gives the label directly.

Missing values and alignment by label

p1_align.py

```
import pandas as pd

week_a = pd.Series({"E": 120, "C": 95, "W": 130})
week_b = pd.Series({"E": 110, "W": 125, "X": 40})
print(week_a + week_b)
```

OUTPUT

```
C      NaN
E    230.0
W    255.0
X      NaN
dtype: float64
```

Series line up by **label**, not position. `C` and `X` exist in only one Series, so their sum is `NaN` (missing). Missing is not zero; pandas refuses to guess.

List, NumPy array or Series?

	list	NumPy array	pandas Series
Find a value by	position	position	label **or** position
Maths on every value	loop	vectorised	vectorised
Mixed types	yes	no (one dtype)	one dtype per Series
Missing values	store `None`	awkward	`NaN` built in
Combine two collections	`+` joins them	by position	**aligned by label**

TRY IT YOURSELF

Create a Series `branch_sales` with labels Sandton, Rosebank, Soweto, Midrand and values 125000, 98000, 76000, 88000.

(a) Print Rosebank's sales by label and the third branch by position. (b) Print the branches above 90 000. (c) Print the total and the label of the best branch. (d) Each branch has a target of 100 000: print how far each is above or below it.

Part 2 DataFrames: a labelled table

A **DataFrame** is a table: an **index** of row labels, **named columns**, and each column is a Series with its own dtype. The fastest way to make one is from a dictionary of lists (or a list of dictionaries, the shape from Session 09).

p2_create.py

```python
import pandas as pd

orders = pd.DataFrame({
    "transaction_id": ["TXN-1", "TXN-2", "TXN-3"],
    "hub_code": ["E", "E", "W"],
    "units": [2, 4, 3],
    "unit_price": [10.0, 12.4, 6.8],
})
print(orders)
print(orders.shape)
print(orders.dtypes)
```

OUTPUT

```
  transaction_id hub_code  units  unit_price
0          TXN-1        E      2        10.0
1          TXN-2        E      4        12.4
2          TXN-3        W      3         6.8
(3, 4)
transaction_id        str
hub_code              str
units               int64
unit_price        float64
dtype: object
```

Part	Meaning
index (0, 1, 2)	row labels inside this table; they can change after sorting or dropping rows
columns	named variables; each column is a Series
dtype	how pandas stores each column (`int64`, `float64`, `str`); it decides which operations make sense

p2_records.py

```python
records = [
    {"transaction_id": "TXN-1", "hub_code": "E", "units": 2},
    {"transaction_id": "TXN-2", "hub_code": "W", "units": 5},
]
print(pd.DataFrame(records))
```

OUTPUT

```
  transaction_id hub_code  units
0          TXN-1        E      2
1          TXN-2        W      5
```

A simulated dataset

To try ideas quickly, generate data. A fixed `seed` makes the random numbers repeatable:

p2_simulated.py

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(seed=42)
simulated = pd.DataFrame({
    "day": np.arange(1, 6),
    "units": rng.integers(80, 130, size=5),
    "price": rng.uniform(5, 50, size=5).round(2),
})
print(simulated)
```

OUTPUT

```
   day  units  price
0    1     84  36.38
1    2    118   9.24
2    3    112  48.90
3    4    101  39.25
4    5    101  40.37
```

One bracket or two?

p2_brackets.py

```
print(type(orders["units"]).__name__, orders["units"].shape)
print(type(orders[["units"]]).__name__, orders[["units"]].shape)
```

OUTPUT

```
Series (3,)
DataFrame (3, 1)
```

`df["units"]` gives a **Series**. `df[["units"]]` gives a one-column **DataFrame**: the inner brackets are a *list* of column names.

Grain: what does one row represent?

Before any calculation, decide the **grain**. In the session's files, one row is **one fictional single-category transaction**, identified by `transaction_id`. Two rows sharing a date, a hub or a category are normal, not duplicates. And the index number is the row's position in the table, **not** its identity.

> TRY IT YOURSELF
>
> Build a DataFrame `staff` with columns `employee_id` (E-1, E-2, E-3), `name` (Thandi, Pieter, Ayesha), `department` (Finance, IT, Finance) and `salary` (32000, 41000, 38500). Print its shape and dtypes, then print the `salary` column as a Series and as a one-column DataFrame.

Part 3 Loading data: CSV, Excel, JSON and SQL

Most real data arrives as a file or from a database. pandas has a reader for each.

CSV and Excel

p3_read.py

```
import pandas as pd

week1 = pd.read_csv("S12_Transactions_Week1.csv", dtype={"transaction_id": "string"})
week2 = pd.read_csv("S12_Transactions_Week2.csv", dtype={"transaction_id": "string"})
hubs = pd.read_excel("S12_Hub_Reference.xlsx", sheet_name="Hub Reference",
engine="openpyxl")

print(week1.shape, week2.shape, hubs.shape)
print(hubs)
```

OUTPUT

```
(56, 8) (57, 8) (3, 3)
  hub_code     hub_name                  region
0        E     East Hub  Eastern operating area
1        C  Central Hub  Central operating area
2        W     West Hub  Western operating area
```

`dtype={"transaction_id": "string"}` keeps the ID as text: an identifier is a label even when it contains digits. `read_excel` needs the sheet name because a workbook can hold several sheets.

> **Watch out:** A relative path such as `"S12_Transactions_Week1.csv"` is read from the folder Python is running in. In VS Code that is the opened folder; in a notebook it is the kernel's working folder (`Path.cwd()`). A `FileNotFoundError` names the folder it searched.

JSON

JSON from an API is often nested. `pd.json_normalize` flattens nested dictionaries into columns:

p3_json.py

```
import json
import pandas as pd

with open("S12_WorldBank_Population_Snapshot.json", encoding="utf-8") as f:
    payload = json.load(f)

observations = payload[1]
table = pd.json_normalize(observations)
print(table[["date", "value", "country.value", "indicator.id"]].head(3))
```

OUTPUT

```
   date       value country.value indicator.id
0  2023  1438069596         India  SP.POP.TOTL
1  2022  1425423212         India  SP.POP.TOTL
2  2021  1414203896         India  SP.POP.TOTL
```

For a simple file that is already a list of flat records, `pd.read_json("file.json")` reads it in one line.

Querying a SQL database

Straight from the SQL module: `pd.read_sql_query` runs a query and returns a DataFrame. Here we put Week 1 into a small SQLite database (built into Python) and query it:

p3_sql.py

```
import sqlite3
import pandas as pd

week1 = pd.read_csv("S12_Transactions_Week1.csv")
connection = sqlite3.connect(":memory:")
week1.to_sql("transactions", connection, index=False)

query = """
SELECT hub_code, status, COUNT(*) AS rows
FROM transactions
GROUP BY hub_code, status
ORDER BY hub_code, status
"""
print(pd.read_sql_query(query, connection))
connection.close()
```

OUTPUT

```
  hub_code     status  rows
0        C  Cancelled     2
1        C  Fulfilled    20
2        E  Cancelled     2
3        E  Fulfilled    17
4        W  Cancelled     3
5        W  Fulfilled    11
6        X  Fulfilled     1
```

With MySQL you would connect with a MySQL driver instead of `sqlite3`; the `read_sql_query` line stays the same.

Source	Reader
CSV text file	`pd.read_csv(path)`
Excel workbook	`pd.read_excel(path, sheet_name=...)`
JSON	`pd.read_json(path)` or `pd.json_normalize(records)`
SQL database	`pd.read_sql_query(sql, connection)`
Python list or dict	`pd.DataFrame(data)`

Part 4 First inspection

A file can open and still be dangerous. **Inspect before you calculate.**

Expression	Helps you answer	Does not establish
`df.head()`	what do the first records look like?	that later rows are clean
`df.shape`	how many rows and columns loaded?	how many rows are valid
`df.columns`	what are the exact column labels?	that similar names mean the same
`df.dtypes`	how did pandas store each column?	that each value fits its meaning
`df.info()`	non-null counts and dtypes together	that the non-null values are right
`df.describe()`	summaries of numeric columns	that a column is a sensible measure
`df.isna().sum()`	how many missing per column?	text that still needs converting

p4_head.py

```
print(week1.head(4))
print(week1.dtypes)
```

OUTPUT

```
   transaction_id transaction_date  ... discount_rate     status
0  TXN-260901-001       2026-09-01  ...          0.05  Cancelled
1  TXN-260901-002       2026-09-01  ...          0.10  Fulfilled
2  TXN-260901-003       2026-09-01  ...          0.00  Fulfilled
3  TXN-260901-004       2026-09-01  ...          0.15  Fulfilled

[4 rows x 8 columns]
transaction_id       string
transaction_date        str
hub_code                str
category                str
units                   str
unit_price          float64
discount_rate       float64
status                  str
dtype: object
```

p4_describe.py

```
print(week1.isna().sum())
print(week1.describe())
print("Week 1 dates:", week1["transaction_date"].min(), "to",
week1["transaction_date"].max())
print("Week 2 dates:", week2["transaction_date"].min(), "to",
week2["transaction_date"].max())
```

OUTPUT

```
transaction_id      0
transaction_date    0
hub_code            0
category            0
units               0
unit_price          1
discount_rate       0
status              0
dtype: int64
       unit_price  discount_rate
count   55.000000      56.000000
mean    77.509091       0.076786
std     84.508495       0.076849
min      6.800000       0.000000
25%     15.600000       0.000000
50%     36.800000       0.050000
75%    108.000000       0.150000
max    299.600000       0.200000
Week 1 dates: 2026-09-01 to 2026-09-07
Week 2 dates: 2026-09-03 to 2026-09-14
```

What the inspection reveals

Finding	Evidence	Question it raises
`units` is text (`str`)	one value is the word "three"	how do we convert it, and what happens to the word?
`unit_price` count is 55, not 56	one price is missing	can that order be valued?
Week 2 starts on 3 September	it contains a copied Week 1 record	will we count that transaction twice?

Each oddity is a question for Part 7. `info()` prints a report (and returns `None`); try it too.

Part 5 Selecting, filtering, sorting and other everyday operations

Start with a question, then choose the action. These map directly onto SQL:

Action	pandas	SQL
choose columns	`df[["transaction_id", "status"]]`	`SELECT`
keep rows	`df[df["hub_code"] == "E"]`	`WHERE`
order rows	`df.sort_values("transaction_date")`	`ORDER BY`

Combining conditions with .loc

p5_loc.py

```
matches = week1.loc[
    (week1["hub_code"] == "W") & (week1["category"] == "Home"),
    ["transaction_id", "transaction_date", "unit_price", "status"],
]
print(matches.sort_values("transaction_date"))
```

OUTPUT

```
    transaction_id transaction_date  unit_price     status
2   TXN-260901-003       2026-09-01        36.8  Fulfilled
26  TXN-260904-003       2026-09-04        79.6  Fulfilled
44  TXN-260906-005       2026-09-06        48.2  Fulfilled
52  TXN-260907-005       2026-09-07        96.8  Fulfilled
```

`.loc[rows, columns]`: a Boolean condition chooses rows, a list chooses columns. Wrap **each** comparison in brackets and join with `&` (and), `|` (or), `~` (not). Python's words `and`/`or` do not work row by row. The left column is the original index, kept through the filter.

.loc uses labels; .iloc uses positions

p5_iloc.py

```
print(week1.loc[0, "status"])
print(week1.iloc[0:3, 0:4])
```

OUTPUT

```
Cancelled
   transaction_id transaction_date hub_code    category
0  TXN-260901-001       2026-09-01        E        Home
1  TXN-260901-002       2026-09-01        E  Essentials
2  TXN-260901-003       2026-09-01        W        Home
```

Habit: Use `.iloc` for quick looks. In real analysis refer to columns by **name**: if the file's column order changes, "column 4" silently becomes a different field.

Rename, replace and map

p5_rename.py

```
renamed = week1.rename(columns={"unit_price": "price_usd", "discount_rate": "discount"})
print(renamed.columns.tolist())

week1_labels = week1["hub_code"].map({"E": "East Hub", "C": "Central Hub", "W": "West
Hub"})
print(week1_labels.head(3).tolist())

print(week1["status"].replace({"Fulfilled": "F", "Cancelled": "C"}).head(3).tolist())
```

OUTPUT

```
['transaction_id', 'transaction_date', 'hub_code', 'category', 'units', 'price_usd',
'discount', 'status']
['East Hub', 'East Hub', 'West Hub']
['C', 'F', 'F']
```

`rename` changes column labels. `map` translates every value through a dictionary (codes it does not find become `NaN`, a useful signal). `replace` swaps only the values you list and leaves the rest.

Unique values and counts

p5_unique.py

```
print(week1["category"].unique())
print(week1["category"].nunique())
print(week1["status"].value_counts())
```

OUTPUT

```
<StringArray>
['Home', 'Essentials', 'Electronics']
Length: 3, dtype: str
3
status
Fulfilled    49
Cancelled     7
Name: count, dtype: int64
```

Minimum, maximum, sum, mean and count

p5_stats.py

```
print(week1["unit_price"].min(), week1["unit_price"].max())
print(round(week1["unit_price"].mean(), 2))
print(week1["unit_price"].count(), "non-missing of", len(week1))
```

OUTPUT

```
6.8 299.6
77.51
55 non-missing of 56
```

Deleting a column or rows

p5_drop.py

```
smaller = week1.drop(columns=["discount_rate"])
print(smaller.shape)

without_cancelled = week1[week1["status"] != "Cancelled"]
print(without_cancelled.shape)
```

OUTPUT

```
(56, 7)
(49, 8)
```

`drop(columns=[...])` removes columns. To remove rows, keep the ones you want with a condition; it is clearer than dropping by index number. Neither changes the original DataFrame unless you assign the result back.

TRY IT YOURSELF

TODO 1 (deck). From `week1`, show only **Fulfilled** rows for hub **E**, with the columns `transaction_id`, `transaction_date`, `category`, `status`, sorted by date. How many rows are there? Why is this still not a table of eligible values?

Extra. How many Week 1 transactions are in each category? Which hub codes appear in Week 1 and Week 2 combined?

Part 6 New columns and business rules

Define the metric **before** computing it. The deck's measure is the discount-adjusted pre-tax order value:

THE RULE

```
net_order_value = units * unit_price * (1 - discount_rate)
```

`discount_rate` is a fraction: 0.10 means 10% off. The result is USD per transaction, before tax and shipping. It is not accounting revenue, and Cancelled transactions do not contribute. Worked example, TXN-260901-002: 4 × 12.40 = 49.60, × 0.90 = **USD 44.64**.

Vectorised: one formula, every row

p6_vector.py

```
import pandas as pd

orders = pd.DataFrame({
    "units": [2, 4, 3],
    "unit_price": [10.0, 12.4, 6.8],
    "discount": [0.0, 0.1, 0.15],
})
orders["value"] = orders["units"] * orders["unit_price"] * (1 - orders["discount"])
print(orders)
```

OUTPUT

```
   units  unit_price  discount  value
0      2        10.0      0.00  20.00
1      4        12.4      0.10  44.64
2      3         6.8      0.15  17.34
```

The arithmetic runs down every row, like a spreadsheet formula filled down a column (Session 11's vectorisation, now with column names).

Looping over a column, and applying a function

p6_apply.py

```
for value in orders["value"]:
    print(f"Order value: {value:.2f}")

def size_band(value):
    if value >= 40:
        return "large"
    return "small"

orders["band"] = orders["value"].apply(size_band)
print(orders)
```

OUTPUT

```
Order value: 20.00
Order value: 44.64
Order value: 17.34
   units  unit_price  discount  value   band
0      2        10.0      0.00  20.00  small
1      4        12.4      0.10  44.64  large
2      3         6.8      0.15  17.34  small
```

You *can* loop over a column, and `.apply(function)` runs your own Session 08 function on every value. Prefer vectorised arithmetic when it exists; use `.apply` for rules that need `if` logic.

> **Watch out:** Rule first, then validate and select rows, then calculate. A formula does not make a bad row reportable. After filtering rows, take a `.copy()` before adding a column.

Part 7 Data quality: missing values, duplicates and types

Cleaning is a **decision process**, not a deletion command: find → judge the risk → treat (keep, exclude, convert or ask) → count → verify. The files contain five cases, each needing a different decision.

Case	Record	Decision
Blank price	TXN-260902-007	leave missing; exclude from value; never fill with 0
Same transaction in both files	TXN-260903-008	keep one copy
Words in units	TXN-260906-003 ("three"), TXN-260909-005 ("two")	flag and exclude; keep the original text
Hub code not in lookup	TXN-260904-008 (code X)	exclude from hub value; ask the source
Cancelled status	14 rows	keep in raw; not counted as fulfilled

Missing is not zero

p7_missing.py

```
print(week1["unit_price"].isna().sum())
print(week1.loc[week1["unit_price"].isna(), ["transaction_id", "units", "unit_price",
"status"]])
```

OUTPUT

```
1
    transaction_id units  unit_price     status
14  TXN-260902-007     2         NaN  Fulfilled
```

If we filled that blank with 0, the report would claim the goods sold for zero dollars, which the file never said. Leave it missing and exclude it from the value measure; the raw record stays for follow-up.

Tool	Use it for
`df.isna()` / `.notna()`	find missing values
`df.dropna(subset=["col"])`	drop rows missing in the columns you name (checks nothing else)
`df["col"].fillna(value)`	fill a genuinely optional label, e.g. `"Not recorded"`, never an invented price or code

Stack the two weeks: concatenation

p7_concat.py

```
raw = pd.concat([week1, week2], ignore_index=True)
print(len(week1), "+", len(week2), "=", len(raw))
```

OUTPUT

```
56 + 57 = 113
```

The weekly files hold the same kind of record, so we stack rows. `ignore_index=True` renumbers the index. Stacking does **not** remove repeats, so 113 is not yet the number of transactions.

Duplicates: two different questions

p7_duplicates.py

```
full_repeat = raw.duplicated(subset=week1.columns.tolist(), keep=False)
id_repeat = raw["transaction_id"].duplicated(keep=False)
print("Rows in full repeats:", int(full_repeat.sum()), " Rows sharing an ID:",
int(id_repeat.sum()))
print(raw.loc[full_repeat, ["transaction_id", "transaction_date", "hub_code", "units",
"unit_price", "status"]])

exact_extra = raw.duplicated(subset=week1.columns.tolist(), keep="first")
working = raw.loc[~exact_extra].copy()
print(len(raw), "raw ->", len(working), "distinct; unique IDs:",
working["transaction_id"].is_unique)
```

OUTPUT

```
Rows in full repeats: 2  Rows sharing an ID: 2
     transaction_id transaction_date hub_code units  unit_price     status
23   TXN-260903-008       2026-09-03        W     1         6.8  Fulfilled
112  TXN-260903-008       2026-09-03        W     1         6.8  Fulfilled
113 raw -> 112 distinct; unique IDs: True
```

A **full-row repeat** (every field identical) is a copy to keep once. A **repeated ID with different values** would be a conflict to investigate, not delete. Here they are the same record. `drop_duplicates()` is the one-line version of the same idea.

> **Watch out:** Not a duplicate: two rows that share Home, hub E or a date. Removing every repeated category would be a serious mistake.

Convert types safely

p7_types.py

```
units_checked = pd.to_numeric(working["units"], errors="coerce")
print(working.loc[units_checked.isna(), ["transaction_id", "units"]])

parsed_dates = pd.to_datetime(working["transaction_date"], format="%Y-%m-%d",
errors="coerce")
print("Dates that failed:", parsed_dates.isna().sum())
```

OUTPUT

```
    transaction_id  units
42  TXN-260906-003  three
68  TXN-260909-005    two
Dates that failed: 0
```

`errors="coerce"` turns an unparseable value into missing instead of stopping. Coercion does not repair the fact: a person can read "three", but the source never documented it, so we flag and exclude the row and keep the original text.

Validity is several conditions together

p7_quality.py

```
quality = pd.DataFrame(index=prepared.index)
quality["cancelled"] = prepared["status"].eq("Cancelled")
quality["bad_units"] = prepared["units_num"].isna() | prepared["units_num"].le(0)
quality["bad_price"] = prepared["price_num"].isna() | prepared["price_num"].lt(0)
quality["bad_discount"] = prepared["discount_num"].isna() |
~prepared["discount_num"].between(0, 1)
quality["bad_date"] = prepared["parsed_date"].isna()

print(quality.sum().astype(int))
print("Rows with more than one reason:", int(quality.sum(axis=1).gt(1).sum()))
```

OUTPUT

```
cancelled       14
bad_units        2
bad_price        1
bad_discount     0
bad_date         0
dtype: int64
Rows with more than one reason: 0
```

No single command is called "clean". With the unmatched hub found in Part 8, the exclusion story reconciles:

Stage	Rows	What the number represents
Both CSV files combined	113	records as received, including a copy
After keeping one copy of the repeat	112	distinct transactions, some unsuitable
Excluded: 14 Cancelled + 2 text units + 1 missing price + 1 unmatched hub	−18	kept visible for the audit
After all eligibility rules	**94**	rows allowed into the measure

TRY IT YOURSELF

(a) How many **Cancelled** rows are in `working`, and which hub has the most? (b) Using `pd.to_numeric(..., errors="coerce")`, list Week 2 rows whose `units` would not convert. (c) Explain in one sentence why filling the missing price with 0 would change both the total and the average.

Part 8 Combining tables and grouping

	concat	merge
Adds	more **rows** of the same structure	more **columns**, matched by a key
Question	where do I put another batch?	what extra facts belong to this key?
Main risk	counting a repeated record twice	multiplying or losing rows through the key
SQL equivalent	`UNION ALL`	`JOIN`

A validated left merge

p8_merge.py

```
joined = prepared.merge(
    hubs,
    on="hub_code",
    how="left",
    validate="many_to_one",
    indicator=True,
)
print(len(prepared), "->", len(joined))
print(joined["_merge"].value_counts())
print(joined.loc[joined["_merge"] == "left_only", ["transaction_id", "hub_code",
"hub_name", "region"]])
```

OUTPUT

```
112 -> 112
_merge
both          111
left_only       1
right_only      0
Name: count, dtype: int64
    transaction_id hub_code hub_name region
31  TXN-260904-008        X      NaN    NaN
```

Argument	Meaning
`on="hub_code"`	the key column in both tables
`how="left"`	keep every transaction, so unmatched rows stay visible
`validate="many_to_one"`	many transactions may share a code, but the lookup must have one row per key (error otherwise)
`indicator=True`	adds `_merge`: `both`, `left_only` or `right_only`

p8_inner.py

```
inner = prepared.merge(hubs, on="hub_code", how="inner")
print("left:", len(joined), " inner:", len(inner))
```

OUTPUT

```
left: 112  inner: 111
```

An inner merge silently drops the hub X transaction. It is useful when dropping unmatched rows is the intended question, risky as a first step when the unmatched key is itself a quality finding. `.join()` does a similar lookup using the other table's index; prefer the explicit `merge` for audits.

What if the lookup lists a key twice?

p8_multiply.py

```
faulty_hubs = pd.concat([hubs, hubs[hubs["hub_code"] == "E"]], ignore_index=True)
multiplied = prepared.merge(faulty_hubs, on="hub_code", how="left")
print("Rows:", len(prepared), "->", len(multiplied))
try:
    prepared.merge(faulty_hubs, on="hub_code", how="left", validate="many_to_one")
except pd.errors.MergeError as error:
    print("MergeError:", error)
```

OUTPUT

```
Rows: 112 -> 145
MergeError: Merge keys are not unique in right dataset; not a many-to-one merge

Duplicates in right:
 hub_code
       E ...
```

Every East row matched two lookup rows, so it appeared twice and East's value would double. Guards: check the lookup key is unique, use `validate`, and compare row counts before and after.

The eligible rows and the measure

p8_eligible.py

```
eligible_mask = (
    joined["status"].eq("Fulfilled")
    & joined["units_num"].gt(0)
    & joined["price_num"].ge(0)
    & joined["discount_num"].between(0, 1)
    & joined["parsed_date"].notna()
    & joined["_merge"].eq("both")
)
eligible = joined.loc[eligible_mask].copy()
eligible["net_order_value"] = eligible["units_num"] * eligible["price_num"] * (1 -
eligible["discount_num"])
print(len(eligible), "eligible; total USD", f"{eligible['net_order_value'].sum():,.2f}")
```

OUTPUT

```
94 eligible; total USD 10,474.39
```

`.gt(0)` on a missing value is False, so rows with unusable units or price fail the rule automatically.

Group by: split, apply, combine

p8_groupby.py

```
by_hub = (eligible.groupby("hub_name", as_index=False)
          .agg(transactions=("transaction_id", "size"),
               net_order_value_usd=("net_order_value", "sum")))
print(by_hub.round(2))
```

OUTPUT

```
      hub_name  transactions  net_order_value_usd
0  Central Hub            38              4247.66
1     East Hub            29              3674.36
2     West Hub            27              2552.37
```

Split the 94 rows by hub, **apply** a count and a sum inside each group, **combine** into one row per hub, like a PivotTable or SQL `GROUP BY`.

Function	Answers
`.size()`	how many rows in each group, regardless of missing values
`.count()`	how many non-missing values in a column per group
`.sum()`	the total of the measure
`.mean()` / `.median()`	the average / the middle value per transaction

p8_two_keys.py

```
by_hub_category = (eligible.groupby(["hub_name", "category"], as_index=False)
                   .agg(transactions=("transaction_id", "size"),
                        usd=("net_order_value", "sum")))
print(by_hub_category.round(2))
```

OUTPUT

```
      hub_name     category  transactions      usd
0  Central Hub  Electronics             9  2232.95
1  Central Hub   Essentials            18   993.49
2  Central Hub         Home            11  1021.22
3     East Hub  Electronics             9  2146.16
4     East Hub   Essentials            12   398.00
5     East Hub         Home             8  1130.20
6     West Hub  Electronics             4  1044.60
7     West Hub   Essentials            11   470.42
8     West Hub         Home            12  1037.35
```

Applying several functions to groups

p8_agg.py

```
print(eligible.groupby("hub_name")["net_order_value"].agg(["size", "sum", "mean",
"median"]).round(2))
```

OUTPUT

```
             size      sum    mean  median
hub_name
Central Hub    38  4247.66  111.78   77.56
East Hub       29  3674.36  126.70   71.28
West Hub       27  2552.37   94.53   73.60
```

Grouping by time

p8_time.py

```
daily = eligible.groupby("parsed_date")["net_order_value"].sum()
print(daily.head(4).round(2))

by_weekday = eligible.groupby(eligible["parsed_date"].dt.day_name())
["net_order_value"].sum()
print(by_weekday.round(2).sort_values(ascending=False).head(3))
```

OUTPUT

```
parsed_date
2026-09-01    1211.69
2026-09-02     594.00
2026-09-03     500.68
2026-09-04     477.48
Name: net_order_value, dtype: float64
parsed_date
Sunday      2139.51
Tuesday     1900.42
Saturday    1871.27
Name: net_order_value, dtype: float64
```

A parsed date column groups by day directly. `.dt` gives date parts: `.dt.day_name()`, `.dt.month`, `.dt.year`. With a date index, `.resample("W")` groups by week.

Reconcile: prove the summary did not change the total

p8_reconcile.py

```
row_total = eligible["net_order_value"].sum()
hub_total = by_hub["net_order_value_usd"].sum()
pair_total = by_hub_category["usd"].sum()
print(f"{row_total:,.2f}  {hub_total:,.2f}  {pair_total:,.2f}")
print(np.isclose(row_total, hub_total), np.isclose(row_total, pair_total))
print(by_hub["transactions"].sum() == len(eligible))
```

OUTPUT

```
10,474.39  10,474.39  10,474.39
True True
True
```

Compare unrounded totals with `np.isclose`; round only for display. Reconciliation checks the arithmetic of grouping, not whether the rules were right.

> TRY IT YOURSELF
>
> (a) Using `eligible`, total `units_num` by **category**. (b) Find the hub-category pair with the most transactions. (c) Which date had the highest eligible order value?

Part 9 Exploratory analysis and visualisation

EDA is a loop: ask → summarise → plot → interpret, then back to quality checks when something surprises you. It describes; it does not prove causes.

Numbers first

p9_describe.py

```
print(eligible["net_order_value"].describe().round(2))
```

OUTPUT

```
count     94.00
mean     111.43
std      113.55
min        6.80
25%       35.81
50%       73.60
75%      136.00
max      599.20
Name: net_order_value, dtype: float64
```

The **median** (73.60) is the middle value: half the orders are below it. The **mean** (111.43) is pulled up by a few large orders. A mean well above the median is a clue to a long upper tail.

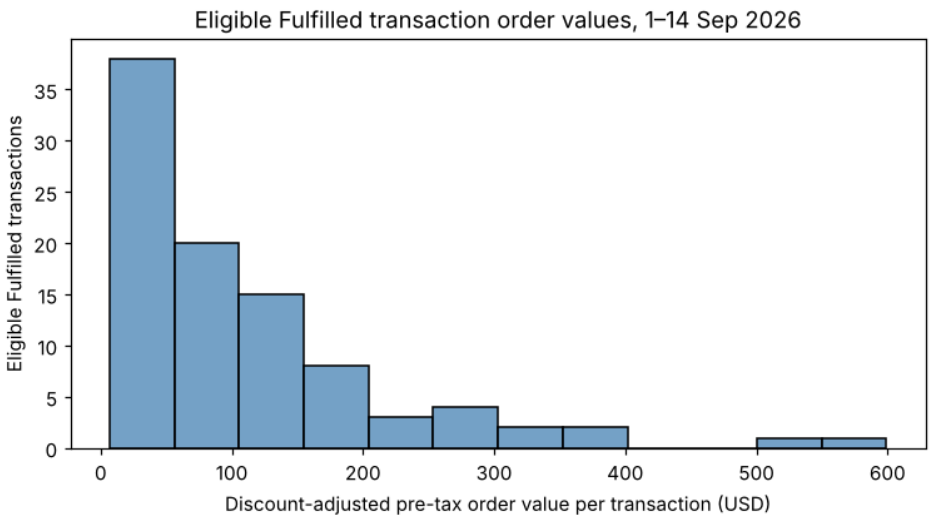
Histogram: one numeric column

p9_hist.py

```
fig, ax = plt.subplots(figsize=(7, 4))
sns.histplot(data=eligible, x="net_order_value", bins=12, ax=ax, color="steelblue")
ax.set(title="Eligible Fulfilled transaction order values, 1–14 Sep 2026",
       xlabel="Discount-adjusted pre-tax order value per transaction (USD)",
       ylabel="Eligible Fulfilled transactions")
fig.tight_layout()
fig.savefig("hist_order_values.png", dpi=110)
plt.show()
```

OUTPUT

```
(nothing is printed)
```



Bar height counts transactions, not USD. Most values sit in the first two bars; the shape is right-skewed.

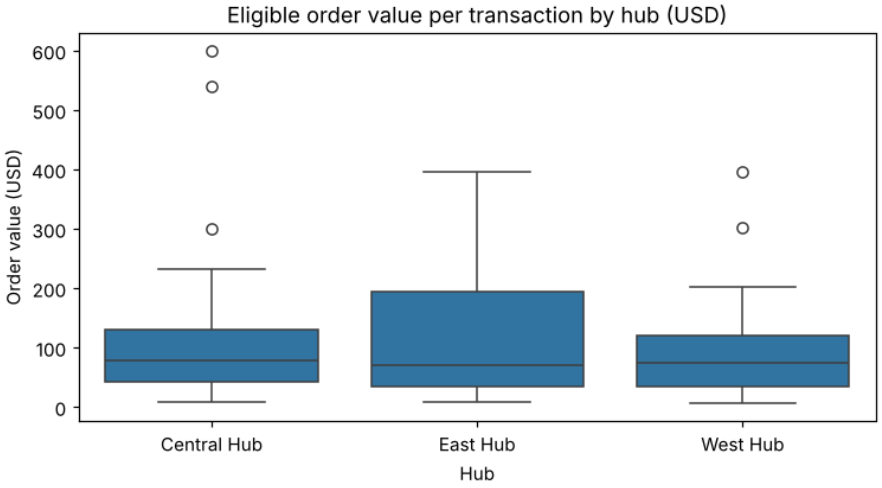
Boxplot: compare groups

p9_box.py

```
fig, ax = plt.subplots(figsize=(7, 4))
sns.boxplot(data=eligible, x="hub_name", y="net_order_value", ax=ax, order=["Central Hub",
"East Hub", "West Hub"])
ax.set(title="Eligible order value per transaction by hub (USD)", xlabel="Hub",
ylabel="Order value (USD)")
fig.tight_layout()
fig.savefig("box_hubs.png", dpi=110)
```

OUTPUT

```
(nothing is printed)
```



Box: middle half of values. Line: median. Dots: values beyond the whiskers, flagged for a look, not errors to delete.

Same data, different question

p9_ranks.py

```
summary = eligible.groupby("hub_name")["net_order_value"].agg(total="sum", mean="mean",
median="median")
print(summary.round(2))
```

OUTPUT

```
               total    mean  median
hub_name
Central Hub  4247.66  111.78   77.56
East Hub     3674.36  126.70   71.28
West Hub     2552.37   94.53   73.60
```

By **total**: Central first (it has the most transactions). By **mean**: East first. By **median**: Central, West, East. A higher mean does not mean a larger typical order.

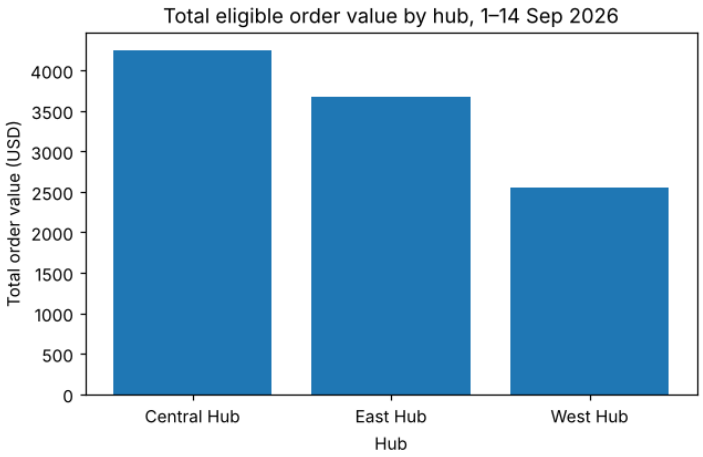
> **Watch out:** seaborn's `barplot` summarises each category with the **mean** by default, not the total. To chart totals, aggregate first and plot the summary table, or say "average" in the title.

p9_bar.py

```
fig, ax = plt.subplots(figsize=(6, 4))
ax.bar(summary.index, summary["total"])
ax.set(title="Total eligible order value by hub, 1–14 Sep 2026", xlabel="Hub",
ylabel="Total order value (USD)")
fig.tight_layout()
fig.savefig("bar_totals.png", dpi=110)
```

OUTPUT

```
(nothing is printed)
```



Aggregate first, then plot: each bar is a sum, and the title says so.

Chart habits

Check	Weak version	Better version
Title	"Order values"	"Eligible order value per transaction, 1–14 Sep 2026 (USD)"
Axis labels	column names like `net_order_value`	plain words with units: "Order value (USD)"
Measure	total and mean mixed	say whether each bar is a sum, a mean or a count
Bars	an axis starting above zero	bars start at zero so lengths compare
Population	silent about exclusions	note "94 eligible Fulfilled transactions"

TRY IT YOURSELF

TODO 10 (deck). Redraw the histogram with `bins=6` and then `bins=24`. What changed in the picture, and what did not change in the data? Write one sentence a manager could read: the pattern, the population and one limit.

Part 10 Working with APIs

Some data is not in a file you were handed; it is **requested** from a service. An **API** is the agreed set of requests and responses between programs. Data comes fresh from its owner in a structured form (JSON), but the response can be late, incomplete, rate-limited or changed.

Anatomy of a request

REQUEST URL

```
https://api.worldbank.org/v2/country/IND/indicator/SP.POP.TOTL?
date=2014:2023&per_page=50&format=json
```

Part	Here	Meaning
Host	`api.worldbank.org`	which server answers
Path	`/v2/country/IND/indicator/SP.POP.TOTL`	which resource: India, total population
Query parameters	`date`, `per_page`, `format`	options after `?`, joined by `&`

Method	Means	Data travels in
GET	"please give me this resource"; should not change anything	the URL (query parameters)
POST	"here is data, process it"	the request body, often JSON

Status family	Meaning	What you do
2xx	success (200 OK)	still inspect the body
3xx	redirect	`requests` follows these for you
4xx	problem with the request (400, 401, 403, 404, 429)	fix the address, key or rate
5xx	problem on the server (500, 503)	wait and retry

One careful request

p10_get.py

```python
import requests

API_URL = "https://api.worldbank.org/v2/country/IND/indicator/SP.POP.TOTL"
API_PARAMS = {"format": "json", "date": "2014:2023", "per_page": 50}

response = requests.get(API_URL, params=API_PARAMS, timeout=(10, 25))
print("HTTP status:", response.status_code)
response.raise_for_status()
api_payload = response.json()
```

Piece	Why
`params=`	a dictionary becomes the `?date=...&format=...` part, correctly encoded
`timeout=(10, 25)`	wait at most 10 s to connect and 25 s for the reply; without it a call can hang
`raise_for_status()`	stops on 4xx or 5xx so a failure cannot slip into the analysis
`.json()`	parses the body into Python lists and dictionaries; it does not check the content

> **Watch out:** An API key is a password for programs. Never put one in a URL, a cell or a shared notebook; read it from an environment variable. The World Bank API needs no key.

Reading the response (saved snapshot)

The class has a saved response, `S12_WorldBank_Population_Snapshot.json`, for when the network is unavailable. The checks are identical either way:

p10_snapshot.py

```
import json

with open("S12_WorldBank_Population_Snapshot.json", encoding="utf-8") as f:
    api_payload = json.load(f)

metadata, observations = api_payload
print(type(api_payload).__name__, len(api_payload))
print(metadata)
print(len(observations), "observations")
print(observations[0])
```

OUTPUT

```
list 2
{'page': 1, 'pages': 1, 'per_page': 50, 'total': 10, 'sourceid': '2', 'lastupdated':
'2026-10-08'}
10 observations
{'indicator': {'id': 'SP.POP.TOTL', 'value': 'Population, total'}, 'country': {'id': 'IN',
'value': 'India'}, 'countryiso3code': 'IND', 'date': '2023', 'value': 1438069596, 'unit':
'', 'obs_status': '', 'decimal': 0}
```

Element 0 is a **dictionary about the response** (page, pages, total, last updated); element 1 is a **list of observations**, one per year. `pages = 1` and `total = 10` mean all ten records fit in this one response. Note that `date` is text ("2023") and `value` is a number that could be null.

Do not trust the first success

p10_checks.py

```
assert metadata["page"] == 1 and metadata["pages"] == 1
assert metadata["total"] == len(observations)
assert all(row["countryiso3code"] == "IND" for row in observations)
assert all(row["indicator"]["id"] == "SP.POP.TOTL" for row in observations)
years = sorted(int(row["date"]) for row in observations)
print("Years:", years[0], "to", years[-1], "| distinct:", len(set(years)))
print("Null values:", sum(row["value"] is None for row in observations))
```

OUTPUT

```
Years: 2014 to 2023 | distinct: 10
Null values: 0
```

A status 200 and valid JSON can still hide an error message, a missing page, the wrong country, or values in another unit. Check structure, pagination and identity before you analyse.

From JSON to a table to a chart

p10_table.py

```
year_population = pd.DataFrame(observations)[["date", "value"]].rename(
    columns={"date": "year", "value": "population"})
year_population["year"] = pd.to_numeric(year_population["year"])
year_population["population"] = pd.to_numeric(year_population["population"])
year_population = year_population.sort_values("year").reset_index(drop=True)
print(year_population)
```

OUTPUT

```
   year  population
0  2014  1312277191
1  2015  1328024498
2  2016  1343944296
3  2017  1359657400
4  2018  1374659064
5  2019  1389030312
6  2020  1402617695
7  2021  1414203896
8  2022  1425423212
9  2023  1438069596
```

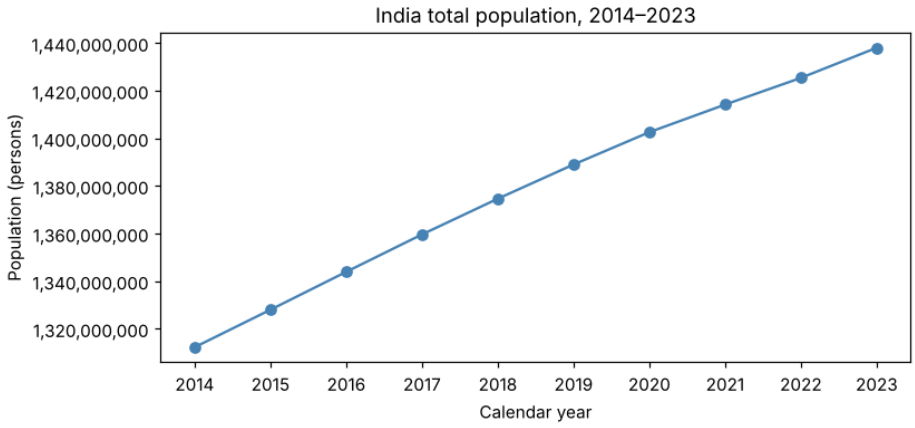
p10_chart.py

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(8.5, 4.4))
ax.plot(year_population["year"], year_population["population"], marker="o",
color="steelblue")
ax.set(title="India total population, 2014–2023", xlabel="Calendar year",
ylabel="Population (persons)")
ax.set_xticks(year_population["year"])
ax.yaxis.set_major_formatter("{x:,.0f}")
fig.subplots_adjust(bottom=0.27, left=0.18)
fig.text(0.01, 0.06, "Source: World Bank, World Development Indicators, SP.POP.TOTL (CC BY
4.0).", fontsize=8)
fig.text(0.01, 0.025, "Offline snapshot captured 2026-10-17 09:11 UTC", fontsize=8)
fig.savefig("population_line.png", dpi=110)
```

OUTPUT

```
(nothing is printed)
```



Source: World Bank, World Development Indicators, SP.POP.TOTL (CC BY 4.0).
Offline snapshot captured 2026-10-17 09:11 UTC

The chart credits its source and says honestly that it used the saved snapshot, not a live request.

Supports: a rise of about 125.8 million people from 2014 to 2023. **Does not:** explain why, or predict what happens next.

Consuming versus exposing an API

	Consume	Expose
Your program is	the client: it asks someone else's service	the server: it answers other programs
Today	asking the World Bank for population data	preview only; built in Session 33
Your concerns	timeouts, limits, changed formats, licence	clear inputs, stable outputs, validation, security

p10_fastapi_preview.py

```python
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
async def root():
    return {"message": "Hello World"}
```

Read only, not run today: answer a GET at the root address with a JSON reply. The same vocabulary (method, address, reply) seen from the other side.

Part 11 Practical checkpoints

> **Trainer note:** The final 34 minutes belong to learners. Use the hint ladder: restate the question, locate the table or JSON element, inspect the last error line and the shape, point to the Starter's hint. Do not project the Solution.

TODO	You write	Check against
1	Fulfilled East rows, four columns, sorted by date	every row hub E and Fulfilled
2–3	stack the weeks; a deduplicated working copy	113 raw → 112 working
4–5	safe conversions; quality reason columns	2 units, 1 price, 0 dates missing; reasons 14 / 2 / 1 / 0 / 0
6–7	validated left merge; eligibility filter with `.copy()`	112 = 111 both + 1 left_only; 94 eligible
8–9	`net_order_value`; hub and hub-by-category summaries	38 / 29 / 27; USD 10,474.39 reconciles
10	labelled histogram	bars add up to 94; right-skewed
11–12	GET with timeout and status check; structure and identity checks	page 1 of 1, total 10, India, SP.POP.TOTL
13–14	year/population table; labelled chart with source credit	10 rows ascending; credit says live or snapshot

Reference results

p11_reference.py

```
print("Raw / distinct / eligible:", len(raw), len(working), len(eligible))
print(eligible.groupby("hub_name").agg(transactions=("transaction_id", "size"),
                                       usd=("net_order_value", "sum")).round(2))
print(f"Total USD {eligible['net_order_value'].sum():,.2f}")
```

OUTPUT

```
Raw / distinct / eligible: 113 112 94
             transactions      usd
hub_name
Central Hub            38  4247.66
East Hub               29  3674.36
West Hub               27  2552.37
Total USD 10,474.39
```

Counts to remember: **113 raw · 112 distinct · 111 matched · 94 eligible**. Eligible total **USD 10,474.39** = 4,247.66 + 3,674.36 + 2,552.37.

Close

Self-check

1. Why is 112 not the number of orders in the hub report, and what is?
2. What is the difference between a left merge and an inner merge, and what could go wrong with a duplicated lookup key?
3. Why can Central have the largest total while East has the largest mean?
4. A request returns status 200. Give two reasons you still would not trust the data yet.
5. What must the credit line on an API chart say?
6. When would you use a Series rather than a list or a NumPy array?

Next: dashboards (Sessions 13–14) reuse these prepared, reconciled tables; statistical inference (15–16) adds evidence standards to EDA; machine-learning preparation (18) reuses the quality rules.

Appendix: Answers to the exercises

Part 1

a1.py

```
import pandas as pd

branch_sales = pd.Series({"Sandton": 125000, "Rosebank": 98000, "Soweto": 76000,
"Midrand": 88000})
print(branch_sales["Rosebank"], branch_sales.iloc[2])
print(branch_sales[branch_sales > 90000])
print(branch_sales.sum(), branch_sales.idxmax())
print(branch_sales - 100000)
```

OUTPUT

```
98000 76000
Sandton     125000
Rosebank     98000
dtype: int64
387000 Sandton
Sandton     25000
Rosebank    -2000
Soweto     -24000
Midrand    -12000
dtype: int64
```

Part 2

a2.py

```
import pandas as pd

staff = pd.DataFrame({
    "employee_id": ["E-1", "E-2", "E-3"],
    "name": ["Thandi", "Pieter", "Ayesha"],
    "department": ["Finance", "IT", "Finance"],
    "salary": [32000, 41000, 38500],
})
print(staff.shape)
print(staff.dtypes)
print(staff["salary"])
print(staff[["salary"]])
```

OUTPUT

```
(3, 4)
employee_id      str
name             str
department       str
salary         int64
dtype: object
0    32000
1    41000
2    38500
Name: salary, dtype: int64
   salary
0   32000
1   41000
2   38500
```

Part 5

a5.py

```
east_fulfilled = week1.loc[
    week1["status"].eq("Fulfilled") & week1["hub_code"].eq("E"),
    ["transaction_id", "transaction_date", "category", "status"],
].sort_values("transaction_date")
print(len(east_fulfilled))
print(east_fulfilled.head())

print(week1["category"].value_counts())
print(sorted(pd.concat([week1, week2])["hub_code"].unique()))
```

OUTPUT

```
17
    transaction_id transaction_date     category     status
1   TXN-260901-002       2026-09-01   Essentials  Fulfilled
4   TXN-260901-005       2026-09-01  Electronics  Fulfilled
5   TXN-260901-006       2026-09-01  Electronics  Fulfilled
11  TXN-260902-004       2026-09-02         Home  Fulfilled
12  TXN-260902-005       2026-09-02   Essentials  Fulfilled
category
Essentials     23
Home           17
Electronics    16
Name: count, dtype: int64
['C', 'E', 'W', 'X']
```

Still not eligible: a Fulfilled row can have text units, a missing price or an unknown hub code. Filtering is not cleaning.

Part 7

a7.py

```
cancelled = working[working["status"] == "Cancelled"]
print(len(cancelled))
print(cancelled["hub_code"].value_counts())

week2_units = pd.to_numeric(week2["units"], errors="coerce")
print(week2.loc[week2_units.isna(), ["transaction_id", "units"]])
```

OUTPUT

```
14
hub_code
C    7
W    4
E    3
Name: count, dtype: int64
    transaction_id units
12  TXN-260909-005   two
```

(c) The order would count as a USD 0 sale: the total gains nothing from it, but the number of transactions rises by one, so the average falls. And the report would claim a zero-dollar sale the source never recorded.

Part 8

a8.py

```
print(eligible.groupby("category")["units_num"].sum())

pairs = eligible.groupby(["hub_name", "category"]).size()
print(pairs.idxmax(), pairs.max())

daily = eligible.groupby("parsed_date")["net_order_value"].sum()
print(daily.idxmax().date(), round(daily.max(), 2))
```

OUTPUT

```
category
Electronics     32.0
Essentials     126.0
Home            56.0
Name: units_num, dtype: float64
('Central Hub', 'Essentials') 18
2026-09-05 1400.68
```

Part 9

Changing `bins` changes how values are grouped into bars, so small bumps can appear or vanish; the 94 values, their median (73.60) and mean (111.43) do not change. *Example sentence:* "Within the 94 eligible orders, most are below about USD 106; the chart does not explain why the largest ones are large."